Лабораторная работа №08

Использование внеэкранных буферов кадра (FBO).

Целью лабораторной работы №8 является реализация эффектов пост-обработки изображения с использованием внеэкранных буферов кадра и механизма рендеринга в текстуру. Внеэкранные буферы кадра (FBO) также необходимо использовать для реализации алгоритма сглаживания ребер полигонов - multisampling anti-aliasing (MSAA).

Использование внеэкранных буферов кадра (FBO).

Во всех предыдущих лабораторных работах, в том числе в проекте предыдущего семестра, рендеринг изображения осуществлялся в стандартный буфер кадра, связанный с экраном. Если быть точнее, вывод осуществлялся в задний буфер цвета и буфер глубины. После смены переднего и заднего буфера цвета, изображение попадало на экран. В большинстве случаев это как раз то, что требуется от программы. Тем не менее, существуют такие ситуации, которые требуют вывода изображения не на экран, а в другой, внеэкранный буфер кадра для его последующего использования, как правило, в виде текстуры, накладываемой на объект.

В качестве примера использования внеэкранных буферов можно привести пример построение отражения в зеркале заднего вида автомобиля. В этом случае камера разворачиваются назад, во внеэкранном буфере кадра строится изображение того, что находится за автомобилем, а затем полученное изображение используется в качестве текстуры и накладывается на модель зеркала при построении изображения уже в основном буфере кадра (в том буфере, который выводится на экран):

Другой сферой применения внэкранных буферов кадра является постобработка изображения, смысл которой заключается в том, что сцена предварительно выводится во внеэкранный буфер кадра, а затем полученное изображение используется для вывода на экран, но с применением каких-либо фильтров, например, для размытия движущихся объектов или объектов, находящихся не в фокусе объектива. Процесс постобработки будет рассмотрен в следующем разделе и его простейший вариант, заключающийся в модификации цветов исходного изображения, необходимо реализовать в данной лабораторной работе.

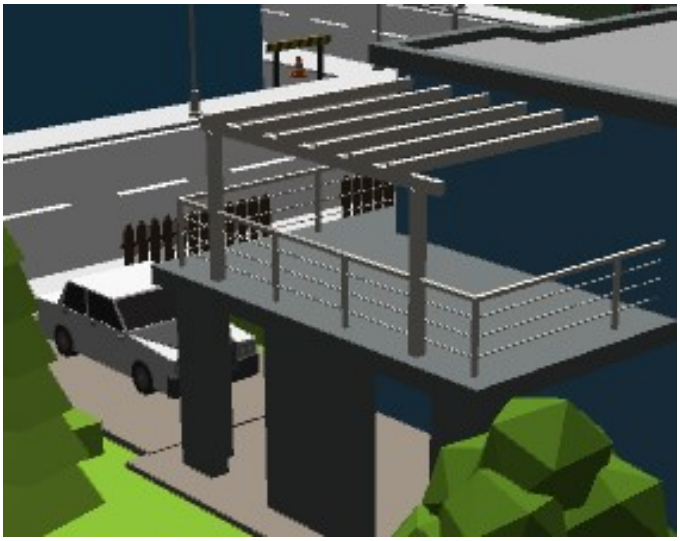
В современной версии OpenGL работа с внеэкранными буферами кадра реализуется с помощью механизма Frame Buffer Object (FBO). FBO – это внеэкранный буфер кадра, связанный с одной или несколькими текстурами. Чаще всего буфер FBO связан с одной текстурой цвета и с одной текстурой глубины, которая необходима, прежде всего, для корректного вывода объектов с учетом их дистанции до наблюдателя, то есть для осуществления теста глубины.

При выборе какого-либо буфера FBO в качестве текущего, вывод изображения будет осуществляться не в задний буфер цвета, как это происходило ранее, а в текстуры, связанные с текущим буфером FBO. Эти текстуры впоследствии можно использовать для наложения на объект при построении изображения в основном буфере кадра. Данная техника получила название «рендеринг в текстуру».

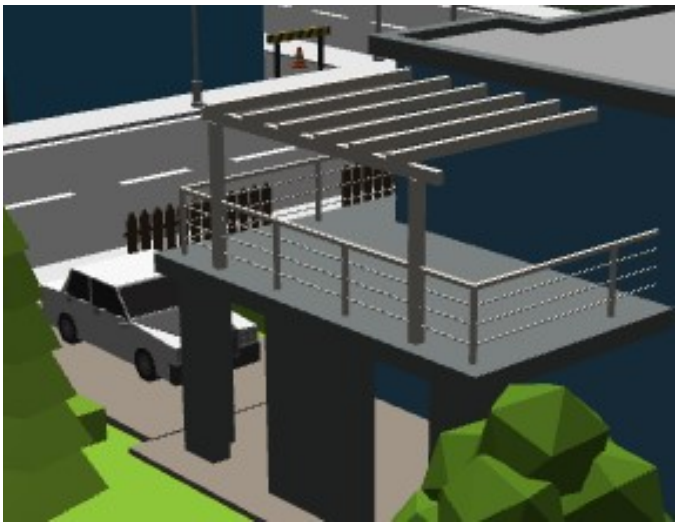
Текстуры, используемые в буфере FBO, могут быть как обычными двухмерными текстурам (тип текстуры при создании - GL_TEXTURE_2D), так и текстурами, поддерживающими множественные выборки - multisampling (тип текстуры при создании - GL_TEXTURE_2D_MULTISAMPLE).

Множественные выборки необходимы для реализации алгоритма сглаживания multisampling anti-aliasing (MSAA). Ниже приведен пример использование в буфере FBO текстур без множественной выборки и с множественными выборками, содержащими 2, 4 или 8 выборок на каждый пиксель буфера кадра:

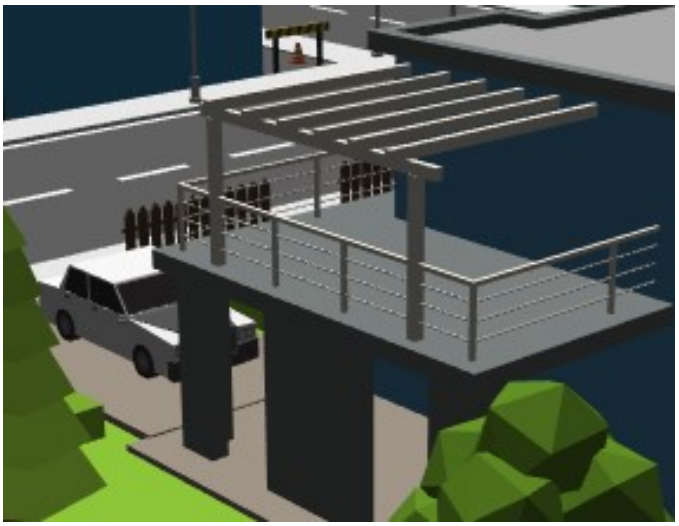
Без множественных выборок



Количество выборок равно 2 (MSAAx2)



Количество выборок равно 4 (MSAAx4)



Количество выборок равно 8 (MSAAx8)

Использование текстур с множественной выборкой (мультисэмплингом) в шейдере является достаточно трудной задачей, поэтому после заполнения такого буфера FBO происходит, как правило, его копирование в другой буфер FBO, построенный на основе обычных текстур. Затем этот буфер FBO может использоваться для последующих этапов, например, для вывода на экран с пост-обработкой. В результате, работу с буфером FBO можно условно разделить на пять этапов:

1. **Создание и инициализация FBO**. Создание FBO подразумевает как создание самого внеэкранного буфера, представленного внутренним объектом OpenGL, так и создание текстур цвета и глубины, которые необходимы для его работы. При этом следует помнить, что текстуры могут быть представлены как обычной двухмерной текстурой, так и текстурой с множественной выборкой.

2. **Выбор FBO в качестве текущего.** Выбор FBO в качестве текущего необходим для того, чтобы указать OpenGL, что все последующие команды вывода объектов будут изменять именно указанный внеэкранный буфер. Следует отметить, что OpenGL различает текущий буфер для записи (то есть буфер, в который будет осуществляться вывод изображения) и текущий буфер для чтения (буфер, из которого данные могут быть считаны).

3. **Возврат к стандартному буферу кадра.** В конечном счете, рано или поздно, необходимо вернуться к стандартному буферу кадра и вывести в него изображение с использованием текстур из ранее заполненного внеэкранного буфера. Именно содержимое стандартного буфера кадра, после смены переднего и заднего буфера цвета, выводится на экран и доступно пользователю.

4. **Использование текстур FBO для наложения на объект.** Основной целью заполнения внеэкранного буфера кадра является получение изображение в текстурах цвета и глубины, связанных с данным буфером FBO. Эти текстуры, по сути, являются обычными двухмерными текстурами и для их последующего использования необходимо иметь возможность привязать их к текстурным блокам. Следует отметить, что использоваться может как текстура цвета, так и текстура глубины. После привязки текстуры к текстурному блоку, её использование ничем не отличается от использования других текстур, например так, как это выполнялось в предыдущих работах.

5. **Копирование одного буфера в другой буфер.** В OpenGL существует возможность копирования содержимого одного буфера FBO в другой буфер FBO. Данная операция называется blitting'ом и выполняется напрямую в видеопамяти (минуя оперативную память), что положительным образом сказывается на скорости выполнения операции. Еще одним преимуществом операции является то, что существует возможность копировать содержимое буфера, использующего текстуры с множественной выборкой, в буфер, использующий обычные текстуры. При этом происходит автоматическое усреднение выборок (resolve) и, соответственно, реализуется сглаживание ребер полигонов. В данной лабораторной работе операция копирования буферов используется именно с этой целью.

Далее приводится более детальное описание каждой из вышеописанных операций с указанием функций OpenGL, необходимых для корректного выполнения этих операций.

Создание и инициализация FBO.

Буфер FBO является внутренним объектом OpenGL, который идентифицируется беззнаковым целым. Для начала работы с буфером FBO его, прежде всего, необходимо создать, что выполняется с помощью функции glGenFramebuffers. Данная функций создает необходимое количество буферов FBO и записывает их идентификаторы в переданный массив. Дополнительную информацию по функции можно найти по следующей ссылке:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glGenFramebuffers.xhtml

Далее необходимо сделать вновь созданный буфер текущим, то есть «привязать» его, используя функцию glBindFramebuffer. Первым параметром является «точка привязки», которая показывает, для чего будет использоваться это буфер. Возможные значения задаются константами и выбираются из следующего списка:

- GL_DRAW_FRAMEBUFFER – буфер будет использоваться для построения изображения, то есть в этот буфер будет осуществляться запись;
- GL_READ_FRAMEBUFFER – буфер будет использоваться для чтения данных (некоторые команды позволяют считывать данные из буфера);
- GL_FRAMEBUFFER – буфер будет использоваться как для построения изображения, так и для чтения данных из него, то есть данная константа является комбинацией двух предыдущих режимов и, чаще всего, используется именно она.

Вторым параметром функции является идентификатор привязываемого буфера. Следует отметить, что многие команды для работы с буферами FBO устанавливают параметры именно того буфера, который является текущим в момент их выполнения. Дополнительную информацию по функции можно найти на официальном сайте по следующей ссылке:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glBindFramebuffer.xhtml

Следующим шагом является создание текстуры цвета, которая будет использоваться в буфере FBO. Данная операция немного отличается в зависимости от того, создается ли текстура без множественной выборки (то есть обычная двухмерная текстура) или же текстура с множественной выборкой (мультисэмплинг текстура).

Для начала рассмотрим процесс создание текстуры цвета без мультисэмплинга. Данный процесс почти полностью совпадает с процессом создания текстурного объекта при загрузке текстуры из внешнего файла. Для этого необходимо создать текстурный объект (glGenTextures), привязать его к текстурному блоку (glBindTexture) и выделить необходимое количество памяти, указав, в том числе, внутренний формат текстуры и её размеры (glTexImage2D). Для лучшего понимания и иллюстрации нескольких ключевых моментов ниже приведен пример кода для создания текстуры цвета:

```
// текстурный объект для цвета
GLuint colorTex;
// создание текстуры цвета
glGenTextures(1, &colorTex);
glBindTexture(GL_TEXTURE_2D, colorTex);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA8, 800, 600, 0, GL_RGBA, GL_UNSIGNED_BYTE, NULL);
```

Прежде всего необходимо обратить внимание на использование функции glTexImage2D, которая, в данном случае, служит не для загрузки массива текселей в текстурный объект, а для указания размера создаваемой текстуры - в примере создается текстура размером 800 на 600 пикселей. Поскольку массив текселей передавать не нужно, последний параметр равен нулевому указателю (NULL), что говорит о том, что копирование пикселей производить не требуется, требуется только выделить необходимое количество памяти.

Также необходимо обратить внимание на задание внутреннего формата текстуры. В данном случае, поскольку создается текстура цвета, используется константа GL_RGBA8, которая говорит о том, что каждый пиксель будет представлен вектором (R, G, B, A) и каждый компонент этого вектора будет иметь размер в восемь бит, что является достаточным для большинства приложений. Стоит отметить, что OpenGL позволяет создавать текстуры цвета с большей точностью представления компонент (например, используя константу GL_RGBA32F), что необходимо, в том числе, для реализации алгоритмов HDR-рендеринга.

Последнее, на что требуется обратить внимание, это на то, что полученный идентификатор текстурного объекта, естественно, необходимо запомнить для последующего использования.

Процесс создания текстуры с мультисэмплингом во многом совпадает с процессом создание обычной текстуры, однако для указания параметров текстуры, прежде всего её размера и внутреннего представления, а также для выделения необходимого количества памяти используется функция glTexImage2DMultisample, а не glTexImage2D. Для иллюстрации некоторых моментов ниже приведен пример кода создания текстуры цвета с мультисэмплингом:

```
// текстурный объект для цвета
GLuint colorTex;
// создание текстуры цвета
glGenTextures(1, &colorTex);
glBindTexture(GL_TEXTURE_2D_MULTISAMPLE, colorTex);
glTexImage2DMultisample(GL_TEXTURE_2D_MULTISAMPLE, 8, GL_RGBA8, 800, 600, false);
```

Здесь, прежде всего, необходимо отметить, что текстура имеет другой тип (GL_TEXTURE_2D_MULTISAMPLE), поэтому во всех функциях, в которых требуется указать тип текстуры (например, в функции glBindTexture) используется именно эта константа.

Выделение памяти для текстуры с мультисэмплингом осуществляется с помощью функции glTexImage2DMultisample, назначение параметров которой является достаточно очевидным. Единственное, на что необходимо обратить внимание, это второй параметр, задающий количество выборок (сэмплов). В приведенном примере количество выборок равно восьми. Чем больше выборок, тем более сглаженными получаются ребра объектов, но тем больше памяти расходуется на хранение текстуры.

Максимальное количество выборок зависит от видеокарты и может быть получено с использованием следующего участка кода:

```
// получение максимального количества выборок
int samples;
glGetIntegerv(GL_MAX_COLOR_TEXTURE_SAMPLES, &samples);
```

Помимо текстуры цвета в большинстве случаев также требуется и текстура глубины. Создание текстуры глубины выполняется аналогично созданию текстуры цвета с единственным отличием, которое заключается в том, что внутренний формат текстуры должен представлять значения глубины, для чего используется соответствующие константы, например: GL_DEPTH_COMPONENT16, GL_DEPTH_COMPONENT24 или GL_DEPTH_COMPONENT32.

Ниже приведен пример создания текстуры глубины без мультисэмплинга:

```
// текстурный объект для глубины
GLuint depthTex;
// создание текстуры глубины
glGenTextures(1, &depthTex);
glBindTexture(GL_TEXTURE_2D, depthTex);
glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT24,
       800, 600, 0, GL_DEPTH_COMPONENT, GL_FLOAT, NULL);
```

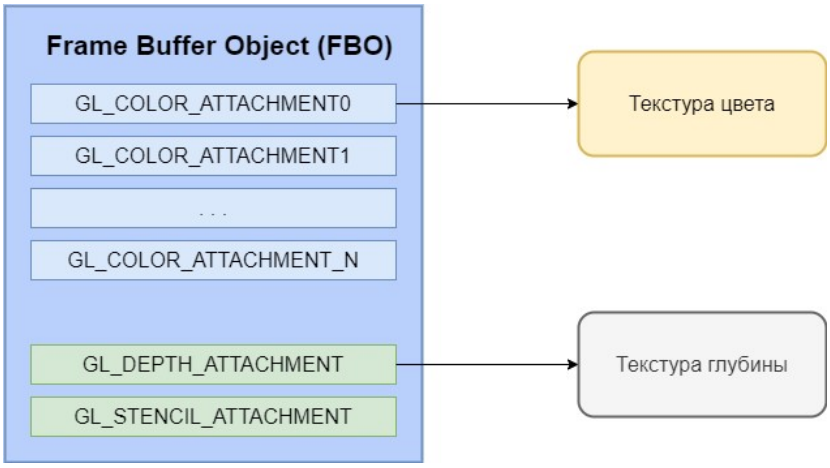
Текстура с мультисэмплингом создается аналогичным образом:

```
// текстурный объект для глубины
GLuint depthTex;
// создание текстуры глубины
glBindTexture(GL_TEXTURE_2D_MULTISAMPLE, depthTex);
glTexImage2DMultisample(GL_TEXTURE_2D_MULTISAMPLE, 8, GL_DEPTH_COMPONENT24,
       800, 600, false);
```

Описание всех функций, необходимых для создания текстуры цвета или текстуры глубины, доступно по следующим ссылкам на официальные страницы консорциума OpenGL:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glGenTextures.xhtml
https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glBindTexture.xhtml
https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glTexImage2D.xhtml
https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glTexImage2DMultisample.xhtml
https://www.khronos.org/opengl/wiki/Image_Format

Следующим шагом, после создания текстуры цвета и текстуры глубины, является их привязка к буферу FBO. Привязка необходима для того, чтобы указать что буфер FBO использует именно эти текстурные объекты. Каждый буфер FBO может быть связан с одним буфером глубины и несколькими буферами цвета, что иногда используется в технике, получившей название Multiple Render Targets. В данной лабораторной работе используется только один буфер цвета и один буфер глубины.



Привязка двухмерных текстур к текущему буферу FBO выполняется с помощью функции glFramebufferTexture2D, имеющий следующий прототип:

void glFramebufferTexture2D(GLenum *target*,
GLenum *attachment*,
GLenum *textarget*,
GLuint *texture*,
GLint *level*);

Первый параметр (target) определяет тип буфера, для которого устанавливается привязка, и может принимать одно из следующих значений: GL_DRAW_FRAMEBUFFER, GL_READ_FRAMEBUFFER или GL_FRAMEBUFFER. Таким образом, привязка происходит к тому буферу, который выбран в качестве текущего буфера указанного типа в момент вызова функции.

Второй параметр (attachment) определяет «точку привязки», то есть указывает будет ли текстура использоваться в качестве текстуры для записи значения глубины фрагмента или для записи его цвета. При этом фрагментный шейдер может формировать не один цвет, а несколько, каждый из которых будет записываться в свою текстуру цвета. В данной лабораторной работе, тем не менее, вычисляется только один цвет, поэтому используется точка привязки GL_COLOR_ATTACHMENT0 для текстуры цвета и точка привязки GL_DEPTH_ATTACHMENT для текстуры глубины.

Третий параметр (textarget) указывает тип текстуры и может принимать одно из двух значений: GL_TEXTURE_2D (привязывается обычная двухмерная текстура) или GL_TEXTURE_2D_MULTISAMPLE (привязывается текстура с мультисэмплингом).

Четвертый параметр (texture) определяет идентификатор текстурного объекта, который будет привязан к буферу FBO в качестве одной из текстур цвета или в качестве текстуры глубины. Следует отметить, что привязываемый текстурный объект должен соответствовать другим параметрам функции glFramebufferTexture2D. Например, если текстурный объект привязывается к точке привязки GL_COLOR_ATTACHMENT0 с указанием типа GL_TEXTURE_2D_MULTISAMPLE, то данная текстура действительно должна быть текстурой цвета с мультисэмплингом.

Последний параметр (level) определяет уровень мипмапа для привязываемой текстуры, что позволяет, например, выводить изображение не в основную текстуру, а в один из её мипмапов. Данная возможность используется достаточно редко, поэтому значение этого параметра, как правило, всегда равно нулю.

Для лучшего понимания, ниже приведен участок кода, демонстрирующий привязку обычной двухмерной текстуры цвета и текстуры глубины к текущему буферу FBO. Привязка текстур с мультисэмплингом осуществляется аналогичным образом:

```
// присоединение текстуры цвета к FBO
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, colorTex, 0);
// присоединение текстуры глубины к FBO
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, depthTex, 0);
```

Дополнительную информацию по функции glFramebufferTexture2D, можно найти на официальном сайте по следующей ссылке:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glFramebufferTexture.xhtml

Приведенная выше последовательность действий позволяет создать буфер FBO с привязанными к нему текстурами цвета и глубины. При этом для того, чтобы буфер FBO считался корректным и его можно было использовать необходимо выполнить определенные условия. Например, в случае использования текстур с мультисэмплингом, необходимо чтобы текстура глубины и текстура цвета содержали одинаковое количество выборок.

Для проверки того, что буфер FBO является корректным, можно воспользоваться функцией glCheckFramebufferStatus:

```
// проверка корректности (completeness status) буфера FBO
GLenum status = glCheckFramebufferStatus(GL_FRAMEBUFFER);
if (status != GL_FRAMEBUFFER_COMPLETE) {
	printf("FBO creation failed\n");
}
```

Дополнительную информацию по функции glCheckFramebufferStatus, а также различные коды ошибок, возвращаемые данной функцией, можно получить по следующей ссылке:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glCheckFramebufferStatus.xhtml

Необходимо отметить, что буферы FBO, используемые для внеэкранного вывода изображения, имеют, в большинстве случаев, размеры, совпадающие с размерами окна приложения. Поэтому, при изменении размеров окна, необходимо, как правило, переинициализировать буферы FBO, что включает в себя удаление старых и создание новых текстур требуемого размера.

Выбор FBO в качестве текущего.

После того, как буфер FBO был успешно создан, можно приступать к его использованию. Первым шагом в использовании буфера FBO является его заполнение, то есть построение в буфере FBO какого-либо изображения. Для это необходимо выбрать буфер в качестве текущего буфера для рендеринга. Данная операция выполняется с помощью функции glBindFramebuffer, которая была рассмотрена ранее.

Стоит отметить, что, в принципе, одна программа может использовать несколько буферов FBO, каждый из которых имеет свой собственный размер (использует текстуры разного размера). Поэтому, после выбора буфера FBO, рекомендуется явным образом указать порт просмотра с помощью команды glViewport так, чтобы порт просмотра совпадал с размерами данного буфера FBO.

Возврат к стандартному буферу кадра.

Рано или поздно необходимо вернуться к стандартному буферу кадра для его заполнения и отображения на экране компьютера, то есть необходимо выбрать стандартный буфер кадра в качестве текущего. Для этого используется функция glBindFramebuffer, где в качестве буфера FBO указывается нулевой буфер.

Не стоит также забывать о необходимости указать порт просмотра для стандартного буфера кадра на случай, если ранее он был изменен для какого-либо другого буфера FBO.

Использование текстур FBO.

Основной целью использования буфера FBO является построение изображения в связанных с ним текстурах для того, чтобы впоследствии использовать эти текстуры для наложения на какой-либо объект, выводимый в стандартный буфер кадра.

С точки зрения OpenGL текстуры, которые ранее были созданы и привязаны к буферу FBO, ничем не отличаются от обычных текстур, которые, например, загружаются из внешних файлов и используются для текстурирования графических объектов. Поэтому, для использования текстур из буфера кадра, необходимо просто привязать их к текстурному блоку, используя функцию glBindTexture, передавая в качестве параметра идентификатор ранее созданного текстурного объекта.

Текстуры буфера FBO, как и прочие текстуры, имеют параметры, отвечающие за режим адресации текселей и режим фильтрации. Тем не менее, текстуры, используемые в буфере FBO, как правило, не имеют мипмапов (поскольку вывод изображения всегда происходит в основную текстуру) поэтому применение для такие текстур режимов фильтрации с использованием мипмапов не только не имеет смысла, но и приведет к неверному расчету цвета текстурируемого объекта.

В результате при привязке текстур из FBO к текстурному блоку рекомендуется использовать следующие параметры:

```
// установка параметров текстуры
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
```

В завершении необходимо отметить, что использовать можно как текстуру цвета (что необходимо реализовать в данной лабораторной работе), так и текстуру глубины (что используется в некоторых алгоритмах, например, для построения теней).

Копирование буферов (blitting).

Последней операцией, которую необходимо реализовать в данной лабораторной работе, является операция копирования одного буфера в другой. Данная операция, помимо прочих вариантов применения, позволяет скопировать буфер с мультисэмплингом в буфер без мультисэмплинга, автоматически учитывая информацию из всех выборок текстуры с мультисэмплингом, то есть осуществив необходимую фильтрацию. Для выполнения операции необходимо:

- Выбрать один из буферов FBO в качестве активного буфера для чтения с помощью функции glBindFramebuffer. Именно из этого буфера будет скопировано изображение.
- Выбрать один из буферов FBO в качестве активного буфера для записи с помощью функции glBindFramebuffer. Именно в этот буфер изображение будет записано.
- Непосредственно скопировать изображение с помощью функции glBlitFramebuffer.

Подробнее про параметры функции можно прочитать по следующей ссылке:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glBlitFramebuffer.xhtml

Класс для работы с буфером FBO.

Для работы с внеэкранным буфером FBO в данной лабораторной работе необходимо создать отдельный класс, структура которого приведена ниже. Особое внимание необходимо обратить на функцию init, которая может создавать как обычные текстуры, так и текстуры, поддерживающие мультисэмплинг, а также на функцию resolveToFbo, которая осуществляет копирование одного буфера FBO в другой:

```
// КЛАСС ДЛЯ РАБОТЫ С FBO
class FBO
{
public:
	// конструктор по умолчанию
	FBO();

	// инициализация FBO (создание самого буфера, текстуры цвета и глубины)
	void init(int w, int h, bool multiSamples = false);

	// выбор FBO в качестве текущего для рендеринга
	void bind();
	// возврат к буферу кадра по умочланию (для вывода на экран)
	static void  unbind();

	// копирование текущего FBO в другой FBO
	void resolveToFbo(FBO& fbo);

	// привязать текстуру цвета к нужному текстурному блоку
	void bindColorTexture(GLenum texUnit = GL_TEXTURE0);
	// привязать текстуру глубины к нужному текстурному блоку
	void bindDepthTexture(GLenum texUnit = GL_TEXTURE1);

private:
	// индекс FBO
	GLuint FBO_Index;
	// текстурный объект для цвета
	GLuint colorTex;
	// текстурный объект для глубины
	GLuint depthTex;
	// прочие параметры
	int width, height, samples;
};
```

Пост-обработка изображения (post-processing).

Пост-обработка (post-processing) – это процесс дополнительной обработки изображения до его вывода на экран, но после того, как была отрисована вся сцена. Пост-обработка применяется для реализации различных эффектов, таких как: bloom, motion blur или depth of field.

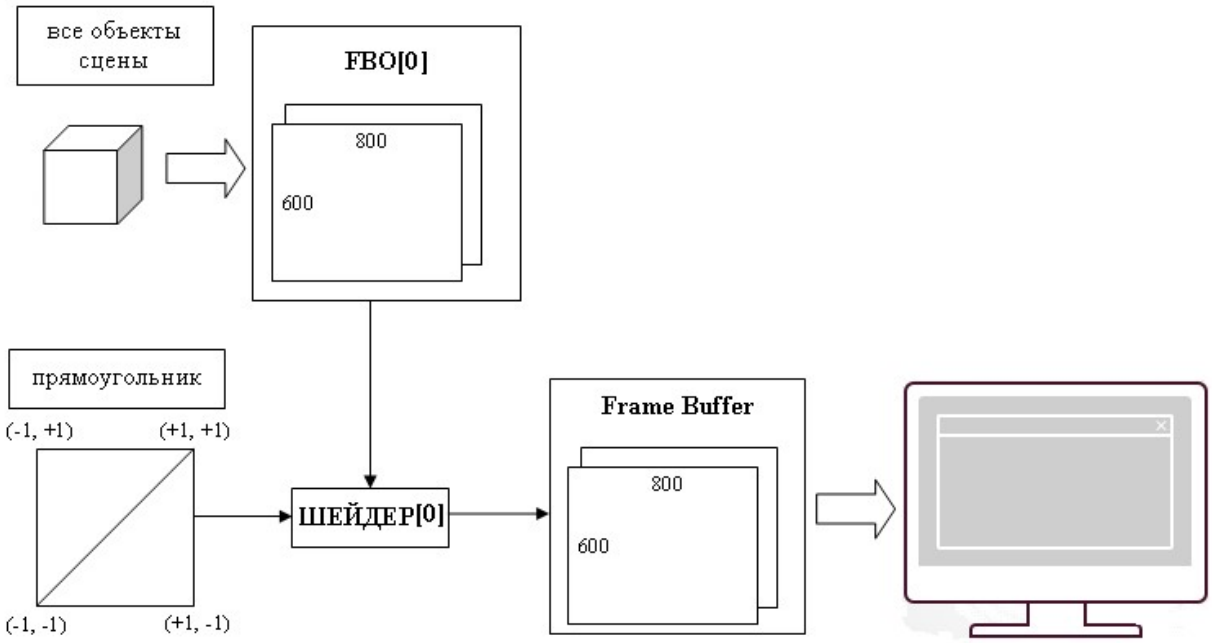
Скриншот, демонстрирующий эффект размытия в движении (motion blur) из игры GTA 5.

Скриншот, демонстрирующий эффект глубины резкости (depth of field) в игре GTA 5.

Постобработка реализуется с помощью рендеринга в текстуру, то есть с использованием буфера FBO. Для этого вся сцена отрисовывается как обычно, но не в стандартный буфер кадра, а во внеэкранный буфер FBO. В результате в текстуре цвета, привязанной к буферу FBO, формируется изображение всей сцены. К полученному изображению необходимо применить тот или иной фильтр, реализующий необходимый эффект, после чего вывести полученное изображение на экран.

Одним из вариантов решения данной задачи является считывание всего массива текселей в оперативную память, преобразование в соответствии с необходимым алгоритмом и передача полученного массива обратно в видеокарту. Данный подход является крайне неэффективным, поскольку требует передачи очень большого объема информации между центральным процессором и видеокартой в каждом кадре.

Другим вариантом решения задачи является использование шейдера для получения итогового изображения. В этом случае, после построения сцены в буфере FBO, полученная текстура используется для наложения на прямоугольник, выводимый в основной буфер кадра. При этом прямоугольник должен занимать всё пространство буфера кадра, а шейдер, используемый для его вывода, должен обращаться к текстуре при вычислении цвета каждого фрагмента. Алгоритм, реализованный в шейдере, определяет применяемый эффект. Схематично, данный процесс может быть представлен следующим рисунком:



Следует отметить, что размер буфера FBO, в который выводится сцена, и буфера кадра (то есть, размер самого окна), как правило, совпадают, поэтому при изменении размера окна необходимо также изменить и размер текстур, используемых в буфере FBO (то есть, провести повторную инициализацию буфера). Изменение размера текстур должно сопровождаться удалением предыдущих текстурных объектов и созданием новых, имеющих необходимые размеры.

В случае, если к исходному изображению применяется только один фильтр (один шейдер), процесс называют однопроходной пост-обработкой (one pass post-processing). С учетом использования буфера с мультисэмплингом (для реализации алгоритма сглаживания MSAA) однопроходная пост-обработка разбивается на следующие этапы:

1. **Выбор буфера FBO с мультисэмплингом.** Постобработка требует, чтобы вся сцена была выведена не в стандартный буфер кадра, а в буфер FBO. Текстуры данного буфера имеют множественную выборку, поэтому полученное изображение будет иметь сглаженные ребра полигонов.

2. **Вывод всей сцены.** Вывод сцены происходит также, как и в предыдущей лабораторной работе, то есть с использованием тех же самых шейдеров, материалов, источников света, камер и текстур (в том числе, с использованием сортировки по материалам и механизма инстансинга). Отличием является то, что теперь объекты будут выводиться не на экран, а в ранее выбранный буфер FBO. Для реализации MSAA необходимо не только использовать буфер с мультисэмплингом, но и включить соответствующий механизм с помощью директивы glEnable(GL_MULTISAMPLE).

3. **Копирование буфера FBO с мультисэмплингом в буфер FBO без мультисэмплинга.** Данный этап необходим для преобразования текстуры с мультисэмплингом в обычную двухмерную текстуру для ей последующего использования.

4. **Выбор стандартного буфера кадра.** Далее происходит возврат к основному буферу кадра. Здесь рекомендуется заново, явным образом, установить порт просмотра для стандартного буфера кадра. С целью оптимизации очистку буфера кадра можно не производить, но в этом случае требуется отключить тест глубины.

5. **Привязка текстуры цвета из внеэкранного буфера FBO к текстурному блоку**. Для вывода полученного изображения сцены в основной буфер кадра необходимо привязать текстуру цвета из буфера FBO без мультисэмплинга, к одному из текстурных блоков. Полученная текстура является обычной двухмерной текстурой и имеет те же параметры, что и все остальные текстуры. Значения параметров можно переопределить в соответствии с реализуемым алгоритмом постобработки, о чем было сказано в предыдущем разделе.

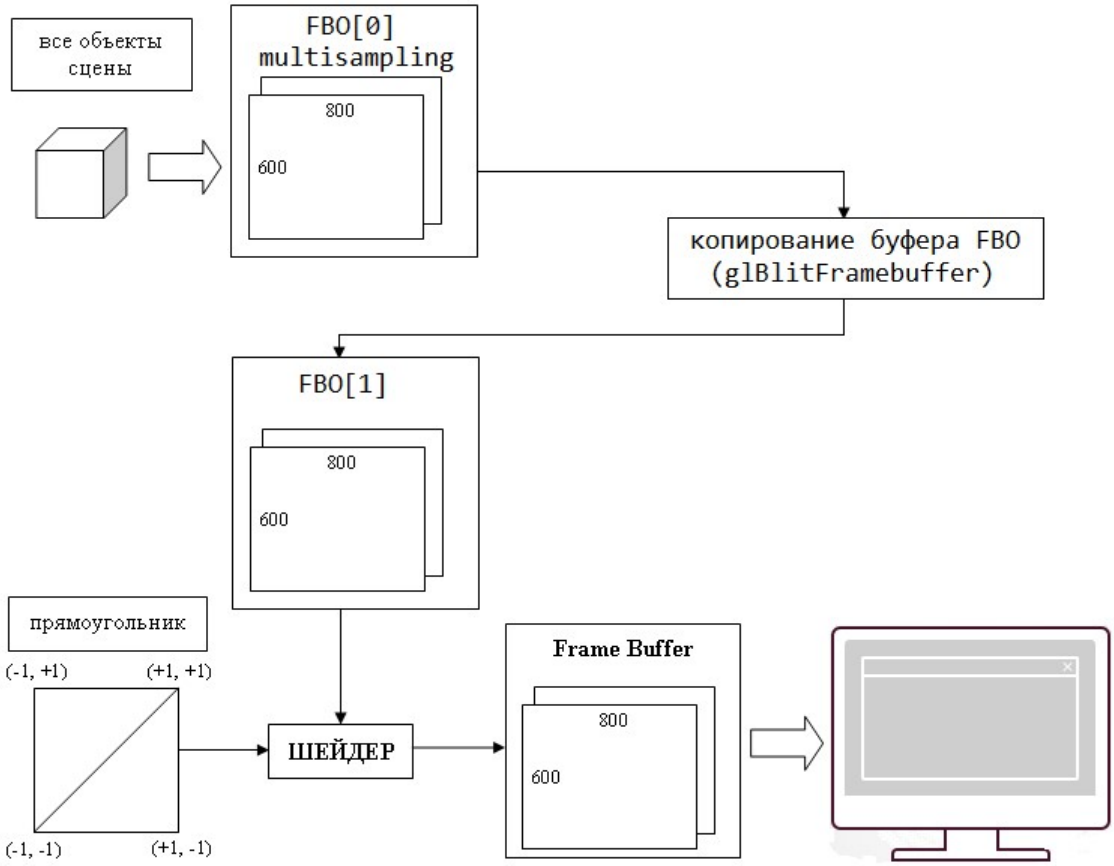
6. **Выбор шейдера, который будет использоваться для постобработки.** Именно выбранный шейдер будет определять, как именно изображение из текстуры будет преобразовываться при выводе на экран. В шейдере необходимо установить значение всех uniform-переменных необходимых для его работы, в том числе, необходимо установить значение переменной типа sampler2D, для определения того, с каким текстурным блоком будет связанна переменная, обеспечивающая доступ к текстуре.

7. **Вывести на экран прямоугольник, занимающий всю площадь экрана.** Для того, чтобы прямоугольник занимал на экране всю площадь, необходимо, чтобы после обработки в вершинном шейдере, координаты (x, y) вершин принимали значения либо -1, либо +1, в соответствии с приведенным рисунком. При этом, значение координаты z не столь важно, поскольку тест глубины в данном случае не используется, но она должна лежать в пределах [-1; +1], для того, чтобы библиотека OpenGL считала, что вершина попадает в область видимости.

   Пример вывода похожего прямоугольника был приведен в первой лабораторной работе, правда там координаты менялись в диапазоне [-0.5; +0.5]. Шейдер, используемый для вывода данного прямоугольника, определяет необходимый эффект постобработки. Пример шейдеров пост-обработки приведен далее.

8. **Поменять местами передний и задний буферы.** В результате выполнения предыдущих шагов в задний буфер кадра был выведен прямоугольник с натянутой на него текстурой. Поскольку текстура, наложенная на этот объект, представляет собой изображение сцены, то складывается впечатление, что именно сама сцена и была выведена на экран. При этом, если шейдер, используемый для наложения текстуры на прямоугольник, каким-либо образом менял значение цвета, полученного из текстуры, то и итоговое изображение будет изменено. В этом и заключается смысл применения шейдеров для постобработки.

Приведенная выше последовательность операций описывает применение однопроходной постобработки, то есть применение к изображению только одного фильтра (одного шейдера). Схема использования буферов FBO и шейдеров показана на следующем рисунке:



Для лучшего понимания ниже приведет полный текст функции finish из класса RenderManager демонстрационного примера. Данный метод отвечает за построение изображения на экране. Несмотря на то, что здесь не приводится описание используемых переменных и вспомогательных функций, алгоритм работы метода должен быть понятен по названию переменных и методов, а также по приведенным комментариям:

```
// завершение вывода кадра
void RenderManager::finish()
{
        // выбираем буфер FBO с мультисэмплингом
        fbos[0].bind();

        // отчищаем буфер кадра
        glClearColor(0.880, 0.890, 0.910, 1.0);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

        // устанавливаем некоторые дополнительные параметры рендеринга
        glEnable(GL_DEPTH_TEST);
        glEnable(GL_MULTISAMPLE);
        glEnable(GL_CULL_FACE);
        glCullFace(GL_BACK);

        // выводим все объекты
        drawAllObjects();

        // копируем буфер с мультисэмплингом в обычный буфер
        fbos[0].resolveToFbo(fbos[1]);

        // переключаемся к стандартному буферу кадра
        FBO::unbind();
        glViewport(0, 0, windowWidth, windowHeight);
        glDisable(GL_MULTISAMPLE);
```

```
        glDisable(GL_DEPTH_TEST);
        glDisable(GL_CULL_FACE);

        // выводим плоскость с наложенной на неё текстурой
        shaders[postProcessingMode + 1].activate();
        shaders[postProcessingMode + 1].setUniform("texture_0", 0);
        fbos[1].bindColorTexture(GL_TEXTURE0);
        drawQuad();
    }
```

В данной лабораторной работе в качестве пост-обработки применяются три разных шейдера: простое копирование текстуры (без обработки), вывод сцены в оттенках серого и использования фильтра sepia, который переводит изображение в желтоватые оттенки, на манер старых, пожелтевших фотографий или газетных вырезок. Переключение между шейдерами, то есть выбор эффекта, в демонстрационном примере осуществляется по нажатию на клавишу «1». Далее кратко будут рассмотрены особенности реализации каждого эффекта.

Стандартный вывод без дополнительной пост обработки.

Первым делом рассмотрим версию шейдера, который никак дополнительно не обрабатывает значение из текстуры, в итоге на экране будет тоже самое изображение, что было получено при выводе самой сцены. Тем не менее, за счет использования буфера кадра с мультисэмплингом будет заметен определенный эффект, в частности, края объектов будут более сглаженными. Полный текст вершинного и фрагментного шейдеров приведен ниже. При рассмотрении шейдера необходимо обратить внимание на три следующих момента:

1. Прежде всего, в демонстрационном примере для вывода прямоугольника используется код из первой лабораторной работы, в котором геометрические координаты x и y каждой вершин лежат в диапазоне [-0.5; +0.5]. В вершинном шейдере, для того чтобы прямоугольник занимал всю площадь экрана, они умножаются на два, в результате чего приводятся к диапазону [-1.0; +1.0].
2. Для доступа к текстуре, как было сказано в одной из предыдущих лабораторных работ, необходимо предоставить текстурные координаты. При этом вовсе не обязательно передавать их в качестве атрибутов вершин, а можно вычислить в самом шейдере, опираясь на какие-либо другие параметры. В частности, в приведенном вершинном шейдере текстурные координаты вычисляются на основании геометрических координат вершин так, чтобы левая нижняя вершина с геометрическими координатами (-0.5, -0.5) имела текстурные координаты (0.0, 0.0), а правая верхняя вершина с координатами (+0.5, +0.5) имела текстурные координаты (1.0, 1.0).
3. Наконец, во фрагментном шейдере необходимо обратиться к текстуре, считать из неё значение по определенным текстурным координатам и, используя это значение, вычислить итоговый цвет фрагмента. В первом варианте шейдера никакие дополнительные преобразования не производятся и цвет фрагмента равен значению из текстуры.

В результате вершинный шейдер выглядит следующим образом:

```
1  #version 330 core
2
3  layout(location = 0) in vec2 vPosition;
4
5  out vec2 texCoord;
6
7  void main ()
8  {
9      texCoord = vPosition + vec2(0.5f, 0.5f);
10     gl_Position = vec4(vPosition * 2.0, 0, 1);
11 }
```

Фрагментный шейдер выглядит так, как показано ниже:

```
#version 330 core

uniform sampler2D texture_0;

in vec2 texCoord;

out vec4 outputColor;

void main (void)
{
    outputColor = texture(texture_0, texCoord);
}
```

Вывод сцены в оттенках серого.

Следующий шейдер переводит исходное цветное изображение в изображение в оттенках серого. Вершинный шейдер для реализации данного алгоритма пост-обработки выглядит также, как и вершинный шейдер предыдущего эффекта, поэтому здесь не приводится.

Разница заключается во фрагментном шейдере, который должен, считав из текстуры значение (r, g, b, a), преобразовать его так, чтобы результирующим цветом стал вектор (lum, lum, lum, 1.0) в котором все компоненты одинаковые и определяются яркостью (luminance) исходного цвета.

Поскольку человеческий глаз лучше различает зеленый цвет и хуже синий, исходные компоненты вносят разное значение в итоговую яркость. В соответствии с рекомендацией ITU BT.709 яркость вычисляется по следующей формуле:

$$luminance = 0.2126 * R + 0.7152 * G + 0.0722 * B.$$

В результате, фрагментный шейдер выглядит так, как показано ниже:

```
#version 330 core

uniform sampler2D texture_0;

in vec2 texCoord;

out vec4 outputColor;

void main (void)
{
    vec4 texColor = texture(texture_0, texCoord);
    float luminance = 0.2126 * texColor.r + 0.7152 * texColor.g + 0.0722 * texColor.b;
    outputColor = vec4(luminance, luminance, luminance, 1.0);
}
```

Sepia-фильтр.

В завершении приводится фрагментный шейдер для перевода исходной картинки в оттенки желтого, что свойственно, например, для старых фотографий. Таким образом, для реализации этого фильтра необходимо сместить исходный цвет в сторону желтого спектра, что осуществляется преобразованием исходного цвета с использованием специально подобранных коэффициентов.

В результате, фрагментный шейдер выглядит так, как показано ниже:

```
#version 330 core

uniform sampler2D texture_0;

in vec2 texCoord;

out vec4 outputColor;

void main (void)
{
    vec4 texColor = texture(texture_0, texCoord);
    outputColor.r= 0.393 * texColor.r + 0.769 * texColor.g + 0.189 * texColor.b;
    outputColor.g = 0.349 * texColor.r + 0.686 * texColor.g + 0.168 * texColor.b;
    outputColor.b = 0.272 * texColor.r + 0.534 * texColor.g + 0.131 * texColor.b;
    outputColor.a = 1.0;
}
```

Задание к лабораторной работе.

Лабораторная работа №8 строится на основе предыдущей работы. К программе предъявляются следующие требования:

1. Необходимо реализовать класс для работы с внеэкранным буфером кадра. Класс должен иметь структуру, описанную в соответствующем разделе и реализовывать все описанные методы.
2. Используя класс для работы с буферами FBO, необходимо реализовать пост-обработку изображения используя три эффекта: стандартный вывод изображения, вывод в оттенках серого и вывод с использованием sepia-фильтра.
3. Переключение между режимами пост-обработки должно выполняться по нажатию на клавишу «1».
4. Для улучшения качества изображения его вывод должен осуществляться с использованием алгоритма сглаживания MSAA, для чего требуется задействовать буфер FBO с мультисэмплингом.
5. В программе должны быть учтены все требования к прошлым лабораторным работам, включая загрузку сцены из внешнего файла, а также алгоритмы оптимизации построения изображения.

Отчет к лабораторной работе должен быть сформирован на основе шаблона, представленного на следующей странице. При оформлении отчета необходимо внести свою фамилию, имя, отчество, номер группы, а также заполнить таблицу. При заполнении таблицы необходимо дать краткое описание реализованным модулям, указать количество строк каждого модуля, а также подсчитать общее количество строк во всем проекте.

Архитектура графических систем (6 семестр)

Выполнил: Иванов Иван Иванович

Группа: 00-00

Проверил: Галибин Сергей Владимирович

Задание:

В лабораторной работе №8 необходимо организовать несколько вариантов постобработки изображения, включая вывод изображения в оттенках серого и применения фильтра sepia.

Защита лабораторной работы:

Для защиты лабораторной работы необходимо знать:

1. **[КГ]** Что такое пост-обработка.
2. **[КГ]** Для чего используются внеэкранные буферы кадра.
3. **[КГ]** Что такое антиалиасинг.
4. **[КГ]** В чем состоит смысл алгоритма MSAA.

Состав проекта:

Название модуля и его назначение	**Количество строк**	
	***.h**	***.cpp**
main - основная программа		
Shader -		
Camera -		
GraphicObject -		
Mesh -		
ResourceManager -		
Texture -		
RenderManager –		
Material -		
Light -		
Scene -		
FBO -		